```
2009 Feb 23 15:42:55 root@localhost Connection ID: 3 Database: test Query: show databases
2009 Feb 23 15:42:55 root@localhost Connection ID: 3 Database: test Query: show tables
2009 Feb 23 15:42:56 root@localhost Connection ID: 3 Database: test Query: create table fo_bawr(ts INTEGER)
2009 Feb 23 15:43:04 root@localhost Connection ID: 3 Database: test Query: create table fo_bawr(i INTEGER)
2009 Feb 23 15:43:08 root@localhost Connection ID: 3 Database: test Query: create table fo_bawr2(i INTEGER)
```

Counting the bytes per-client connection

In the last example, we are going to use the following static probes to write a D script that will give the number of bytes transferred on a per-client connection basis:

- *connection__start(unsigned long conn_id, char *user, char *host);*: This probe is fired when a new client connects to the server
- *command__done(int status)*: This probe is fired when the client disconnects
- These probes can be used to measure the number of bytes transferred in read and write operations:

```
probe net__read__done(int status, unsigned long bytes);
probe net__write__start(unsigned long bytes);
```

The script is as follows:

```
#!/usr/sbin/dtrace
#pragma D option quiet

dtrace::BEGIN
{
printf("Tracking the bytes.. Hit Ctrl-C to end.\n");
}

mysql*::connection-start
{
 self->bytes_read = 0;
 self->bytes_write = 0;
 self->conn_id = arg0;
 self->who = strjoin(copyinstr(arg1),strjoin("@",copyinstr(arg2))); /* Get the username */
 printf("Got a client connection at %Y from %20s with ID %u\n",
walltimestamp, self->who, self->conn_id);
 self->client_connect_start = timestamp;
}
mysql*::net-read-done/* using the mysql provider */
{
 self->bytes_read = self->bytes_read + arg1;
}
mysql*::net-write-start/* using the mysql provider */
{
 self->start_w = timestamp;
 self->bytes_write= self->bytes_write + arg1;
}
mysql*::connection-done
{
printf ("Connection with ID: %u closed.\nTotal Bytes transferred: %u \nTotal connection time (ms): %-9d\n\n",self->conn_id, self->bytes_read + self->bytes_write,(timestamp-self->client_connect_start)/1000000 );
}
```

The above script reports the data transfer activity as follows:

```
Tracking the bytes.. Hit Ctrl-C to end.
Got a client connection at 2009 Feb 23 20:07:24 from root@localhost with ID 50
Got a client connection at 2009 Feb 23 20:07:27 from amit@localhost with ID 51
Connection with ID: 50 closed.
Total Bytes transferred: 1081204027
Total connection time (ms): 17650

Connection with ID: 51 closed.
Total Bytes transferred: 3039908614
Total connection time (ms): 23787
```

Where can DTrace help the MySQL community?

The niche MySQL community into which DTrace can breathe life is the database administrators (DBAs) who strive to keep the database in good health at all points. It's easy to identify the performance bottlenecks that might have crept into the server over a period of time. With intelligent probe descriptions, it's simple to get relevant statistics of a running MySQL server.

Besides the DBAs, DTrace is a great tool to understand how the control flow occurs in the MySQL server, from when it receives a client request, till it serves the request. This makes it very easy to understand all the different sub-components of the MySQL server architecture. END

References and further information

- Tracing mysqld using DTrace: *dev.mysql.com/doc/refman/6.0/en/dba-dtrace-server.html*
- DTrace Community: *opensolaris.org/os/community/dtrace*
- DTrace Architecture: *docs.sun.com/app/docs/doc/819-5488/gcdxn?a=view*
- Solaris Dynamic Tracing Guide: *docs.sun.com/app/docs/doc/817-6223*
- Using DTrace with MySQL (MySQL University Session). In this session Martin MC Brown covers in a lot of details, how you could make use of the static probes in MySQL server, starting MySQL-6.0.8: *forge.mysql.com/wiki/Using_DTrace_with_MySQL*
- Optimising MySQL Database Application Performance with Solaris Dynamic Tracing: *wikis.sun.com/display/BluePrints/Optimizing+MySQL+Database+Application+Performance+with+Solaris+Dynamic+Tracing*

By: Amit K. Saha

The author currently works in MySQL Engineering at Sun Microsystems. For any queries on this article, please feel free to mail him at amitsaha.in@gmail.com

This article was first published in MySQL Developer Zone at dev.mysql.com